Project 1 - Self-Healing Infrastructure

Automated incident detection and application recovery

1. Introduction

Application outages can require manual intervention and extend downtime. This project implements a local demonstration of automated failure detection and recovery for a Dockerized Node.js application.

2. Abstract

Prometheus monitors application health metrics and sends firing alerts to Alertmanager. An authenticated Python Flask webhook validates relevant alerts, checks safeguards, and launches an Ansible playbook to restart the unhealthy Docker container. The recovery process verifies the health endpoint and records outcomes in JSON audit logs.

3. Tools Used

Technology	Purpose
Node.js + Docker	Sample application, failure simulation, and container runtime
Prometheus	Metrics scraping and unhealthy-application alert rules
Alertmanager	Alert routing to authenticated webhook
Python + Flask	Alert validation, cooldown, recovery orchestration, audit logging
Ansible + Ubuntu WSL2	Docker restart automation and post-restart health verification

4. Steps Involved in Building the Project

1. Created a Node.js application exposing /health, /metrics, and failure/recovery simulation endpoints.

2. Containerized the app and configured Prometheus and Alertmanager with Docker Compose.

3. Defined the SelfHealingAppUnhealthy alert using the selfhealing_app_healthy metric.

4. Built an authenticated Flask webhook that accepts relevant firing alerts, checks actual application health, and starts Ansible asynchronously.

5. Implemented an Ansible playbook to restart selfhealing-app and verify that /health reports UP.

6. Added concurrent-run protection, a 120-second cooldown, and persistent JSON audit events.

5. Testing and Results

Verified Prometheus targets and alerts, Alertmanager delivery, manual and automatic Ansible recovery, healthy-app protection, concurrent request rejection, cooldown behavior, successful recovery (return code 0), and simulated recovery failure logging (return code 1). Screenshots are included in the project repository.

6. Conclusion

The working system demonstrates a complete monitor-alert-remediate-verify workflow. It reduces manual recovery effort in a controlled demo and illustrates practical DevOps automation. Production use would require stronger network isolation, TLS, hardened service hosting, and durable coordination.